Computer Architecture
Instructor: Gedare Bloom
Term Project

You must do this assignment in a group. Group member scores may be differentiated. You may not use code downloaded from the Internet without prior permission. Always watch the course website for updates on the assignments.

Read all instructions carefully. **Start early.** I recommend completing one Part each week.

Welcome to the *Foo* Company. You are part of a design team that needs to evaluate the RV32I machine that management has decided to consider. *Foo* is creating some data analysis software and needs to know that RV32I is worthwhile. The primary task of the software is a function that sums the columns of a matrix. The following is the high-level source code for this task:

```
unsigned int mm[nrow][ncol];
unsigned int sum[nrow];

void sum_columns(unsigned int **matrix, int nrow, int ncol) {
  for (int i = 0; i < ncol; i++) {
      sum[i] = 0;
      for (int k = 0; k < nrow; k++) {
          sum[i] += matrix[k][i];
      }
  }
}
```

To make management happy you need to implement this functionality in RISC-V assembly without using the add (or sub) instruction. Instead, you need to use a software emulation of add implemented in a `do_add` function that takes as argument two integers in registers and does the following:

1. Add the two integers without using the 'add*' or 'sub*' RISC-V instruction. This restriction applies to the `sum_columns` function and any functions that do_add calls. You may use addi to adjust the stack pointer if necessary for making function calls. You will likely need to use the 'or', 'and', 'xor', 'sll', and 'srl' instructions.
2. Return the result in a register. If there is an arithmetic overflow, return an error condition identifying that an overflow occurred in another register.

Then evaluate the performance of your program on the RV32I machine and consider several possible optimizations as detailed below. The machine is a fully pipelined processor with hazard detection, data forwarding, and cache. It predicts branches and jumps as never taken, and implements branch and jump instructions in the MEM stage. You have a simulator for the machine, which is the QtRVSim platform.

Set the cache parameters for both i$ (Program Cache) and d$ (Data Cache) to be direct-mapped with block size of 16 bytes and 64 blocks in each cache. Hence, each cache size is 1 KiB.

Group Formation

You may choose your own team member(s). Agree to form a group with 1 to 2 of your classmates, and arrange to join the same Project_Groups in Canvas (People Groups). Do this ASAP, but not later than 2 weeks before the project due date. Send the instructor an e-mail if you have trouble and need a manual adjustment of your group. **DO NOT** join a random group without first discussing with the group members!

I require that you use git source version control. You may use bitbucket or github, with a private git repository that is shared among your team members. You may like to use branches and pull requests to synchronize your work with each other, or you may also work directly on the same repository (e.g., the first team member's) and push directly to a branch (or even just use the main branch) there by giving write access to the other team member. Commit and push frequently as you work with your team.

Part 1: Unsigned Column Sum (20%)

Your first task is to finish the implementation of sum_columns so that it works on unsigned (positive) integers. A simple test case could be

$$A = \begin{bmatrix} 0 & 1 & 2 \\ 1 & 2 & 3 \end{bmatrix}$$

sum_columns(A, 2, 3);

$$sum = \begin{bmatrix} 1 & 3 & 5 \end{bmatrix}$$

Management would like your software implementation to work on test cases up to at least nrow = 32 and ncol = 32.

Part 2: Memory-Aware Implementation (20%)

Rewrite the assembly code you generated for sum_columns to use the following high-level source code structure:

```
unsigned int mm[nrow][ncol];
unsigned int sum[ncol] = {};

void sum_columns(int **matrix, int nrow, int ncol) {
  for (int i = 0; i < nrow; i++) {
     for (int k = 0; k < ncol; k++) {
        sum[k] += matrix[i][k];
     }
  }
```

```
}
```

Part 3: Performance Analysis (20%)

1. Compare your implementations from Parts 1 and 2 using speedup. Consider a variety of matrix sizes from 8x8 up to 256x256. Does one implementation do better than the other? Why do you think that is?
2. Replace your software implementations from Part 1 and Part 2 of addition by the add instruction. Repeat your analysis comparing these two implementations with hardware addition. Do your conclusions change? Why do you think that is?
3. Repeat the analysis of steps 1 and 2 with four times as much data cache. Consider quadrupling the block size, quadrupling the number of blocks, or doubling the block size and doubling the number of blocks. Do your conclusions change? Does one of the cache parameters help more than the other? Why do you think?

If you had to choose between implementing addition in hardware and quadrupling the size of the data cache, which would you do? Why?

Part 4: Report and Log (20%)

Determine how best to represent the results of your analysis from Part 3 using tables or graphs. Management is most impressed by fancy graphics and gets bored by lots of text.

Create a written report in PDF format that explains:

- Your team composition and roles of each member
- Your method of implementation of unsigned column average
- Your analyses of performance
- Your recommendation for adding more cache versus implementing addition in hardware
- Anything else you would like to say to management

Add the report and its source (e.g., docx file) to your repository. Be careful to avoid merge conflicts with binary format files such as Word or PDF because they can be difficult to resolve.

In addition, generate a text file with the contents of your git log using the command:

```
$ git log --pretty=format:"%h %an (%ad): %s" --date=short > log.txt
```

Submit the log.txt file.

Part 5: Presentation (20%)

Prepare a presentation about your project including what you did, how it worked, any design changes you made, your benchmarking and testing results, and any conclusions you have. Some slides with sample output from your program and graphs of any data should be included. The presentation will be submitted with your final package as well as separately, so that all presentations can be downloaded and prepared for smooth

transitions. Do not plan to use your own laptop for the presentation, and do not plan to give a demo. Include visual/descriptive analyses of performance and your recommendation for adding more cache versus implementing addition in hardware. Plan for a 5-minute presentation plus Q&A. You can either use powerpoint or create a PDF.

Submission Instructions (read carefully)

Make a zip file with your group members' UCCS usernames (email address without the @uccs.edu part) as the name.
Put all your files in ${USERNAME1}-${USERNAME2}-project/ directory. For example, I would replace ${USERNAME1} above with gbloom, and ${USERNAME2} with the second member of my team. If your team has a third member, add their username as well. **Do not include compiled output in your submission!** Compress the folder to a file ${USERNAME1}-${USERNAME2}-project.zip. Include the PDF report, powerpoint or PDF presentation, your source code, and the log.txt file. Upload the zip file and separately upload the report and the presentation files to the Project submission on Canvas.